



Data Processing NLP

Presented By: Chutimon Nunthasena 6640201080



การเตรียมข้อมูล

- ใช้ train-test split 80:20
- กำหนด random seed เพื่อให้แบ่งได้ผลลัพธ์เดิมทุกครั้ง
- ใช้ stratified split เพื่อรักษาสัดส่วนของแต่ละคลาส
- ใช้ index นี้เดียวกันในทุกการทดลอง

- ใช้ SVM ตัวเดียวกันทุกการทดลอง
- ใช้ train/test split เดิม
- เปลี่ยนเฉพาะ feature



```
train_idx, test_idx = train_test_split(  
    indices,  
    test_size=0.2,      # แบ่งข้อมูลเป็น train 80% / test 20%  
    random_state=42,   # กำหนดค่า seed  
    stratify=simple_analysis["label"] # รักษาสัดส่วน  
)
```



Baseline Features

Page 03



num_pos
จำนวนคำเชิงบวก



num_neg
จำนวนคำเชิงลบ



num_neu
จำนวนคำเชิงกลาง



num_tokens
จำนวน token ทั้งหมด



unique_tokens_ratio
สัดส่วน token ไม่ซ้ำ



normalized_sentiment_score
คะแนน sentiment
แบบ normalize

- Baseline numeric features (6 ตัว) ที่สร้างจาก token ใน simple_analysis

วัตถุประสงค์:

ให้โมเดลเห็นสัญญาณจากจำนวนคำ/ความยาว/ความหลากหลาย
ของคำที่สัมพันธ์กับอารมณ์

```
vectorizer = CountVectorizer(  
    tokenizer=lambda x: x.split(' '), #กำหนด tokenizer ให้แยก token ตาม '  
    ngram_range=(1,2), # ใช้ทั้ง unigram และ bigram  
    max_features=5000) # จำกัดจำนวนฟีเจอร์ เพื่อควบคุมมิติข้อมูล
```

CODING

=====

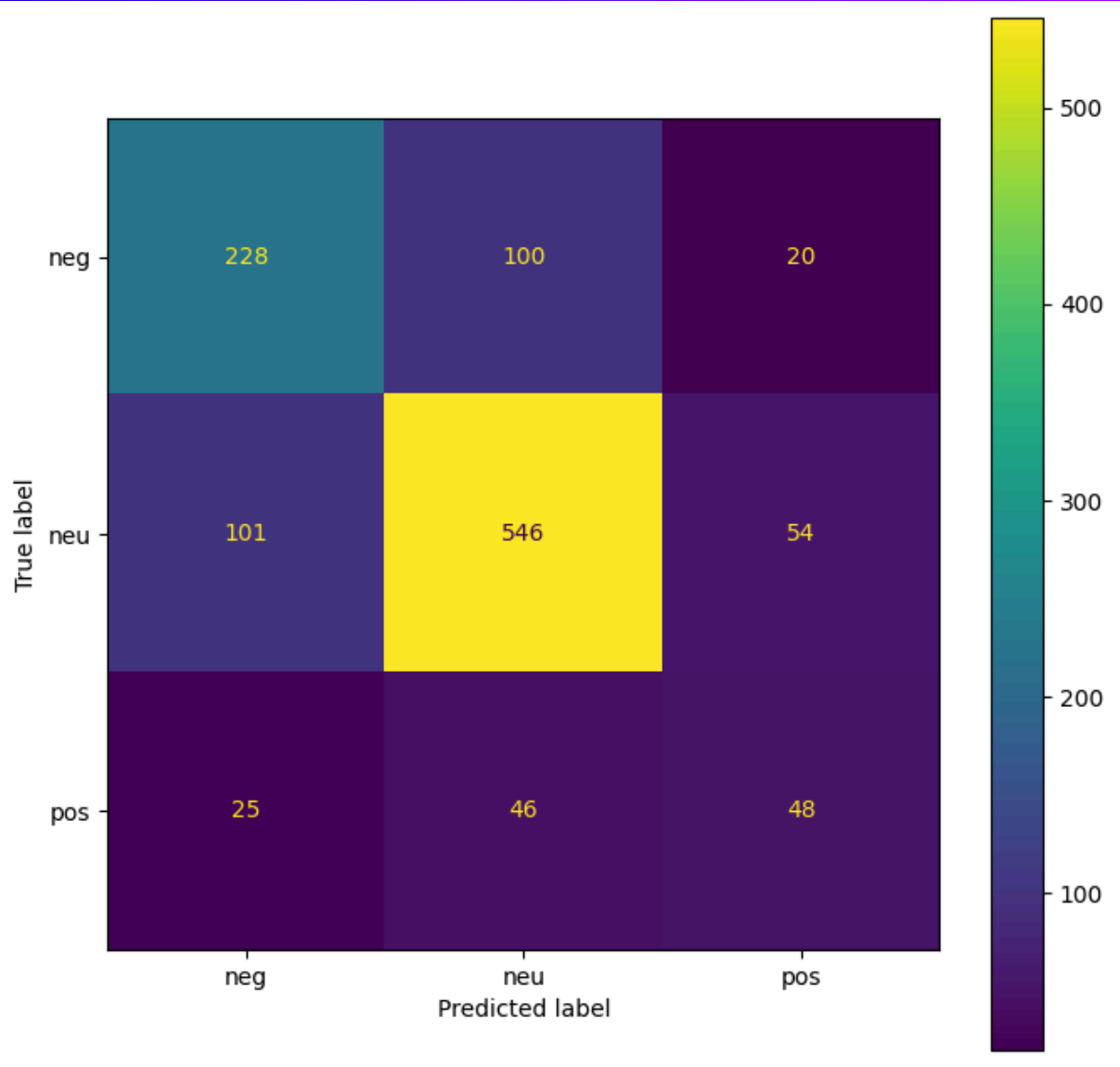
BASELINE

Accuracy = 0.7038 | F1-score = 0.7046

Classification Report:

	precision	recall	f1-score	support
neg	0.644	0.655	0.650	348
neu	0.789	0.779	0.784	701
pos	0.393	0.403	0.398	119
accuracy		0.704		1168
macro avg	0.609	0.612	0.611	1168
weighted avg	0.706	0.704	0.705	1168

=====



ไอเดียที่ 1

Page 05

Emotion Marker Ratio

วัตถุประสงค์ของไอเดียนี้คือ

เพื่อสร้างฟีเจอร์ใหม่ที่สะท้อนระดับความเข้มของอารมณ์ จากรูปแบบการพิมพ์ของผู้ใช้ โดยพิจารณาการใช้เครื่องหมายซ้ำและการลากเสียง ซึ่งมักถูกใช้เพื่อแสดงอารมณ์ในข้อความ เช่น

- เครื่องหมายซ้ำ (!!!, ???)
- การลากเสียง (มากกก, โคตรsss)

???

มากกก

!!!

สร้างฟีเจอร์ emotion_marker_ratio

Page 06

แนวคิด:

- ตรวจสอบ token ที่เป็น emotion marker
- คำนวณสัดส่วนของ token เหล่านี้ต่อจำนวน token ทั้งหมด

สูตร:
$$\text{emotion_marker_ratio} = \frac{\text{จำนวน emotion marker token}}{\text{จำนวน token ทั้งหมด}}$$

สร้างฟีเจอร์ emotion_marker_ratio

Page 07



```
# ตรวจสอบว่า token นี้เป็น 'ตัวบ่งชี้อารมณ์' (emotion marker) หรือไม่ เช่น !!! ??? / ลากเสียง มากกก / โทตsss / 5555
def is_emotion_marker(tok: str) -> bool:
    tok = str(tok) # กันปัญหา tok เป็น NaN หรือชนิดอื่น

    # 1) เครื่องหมายอารมณ์ซ้ำ ๆ เช่น "!!!", "???", "!?!" ถ้า token มีแต่ ! หรือ ? อย่างเดียว (ยาวเท่าไรก็ได้) => True
    if re.fullmatch(r"[!?]+", tok):
        return True

    # 2) ตัวอักษร/ตัวเลขซ้ำติดกัน >= 3 ตัว เช่น "มากกก" (ก ซ้ำ), "โทตsss" (s ซ้ำ), "5555" (5 ซ้ำ), "(.)\1\1+" แปลว่า "มีตัวเดิมซ้ำอย่างน้อย 3 ครั้ง"
    if re.search(r"(\1\1\1+)", tok):
        return True
    return False

# คำนวณสัดส่วนของ emotion marker ต่อจำนวน token ทั้งหมดในข้อความ
def emotion_marker_ratio(text: str) -> float:

    toks = str(text).split("|") # แปลงเป็น string แล้วแยก token ตามตัวคั่น "|" ที่ใช้ใน preprocessing
    toks = [t for t in toks if t != ""] # ลบ token ว่าง ๆ ออก

    # ถ้าไม่มี token เลย ป้อนกลับหาร 0
    if len(toks) == 0:
        return 0.0

    # นับจำนวน token ที่เป็น emotion marker แล้วหารด้วยจำนวน token ทั้งหมด
    return sum(is_emotion_marker(t) for t in toks) / len(toks)
```

=====

IDEA 1: emotion_marker_ratio

Accuracy = 0.7038 | F1-score = 0.7040

Classification Report:

	precision	recall	f1-score	support
neg	0.637	0.661	0.649	348
neu	0.790	0.779	0.784	701
pos	0.397	0.387	0.391	119

accuracy

0.704

1168

macro avg

0.608

0.609

0.608

1168

weighted avg

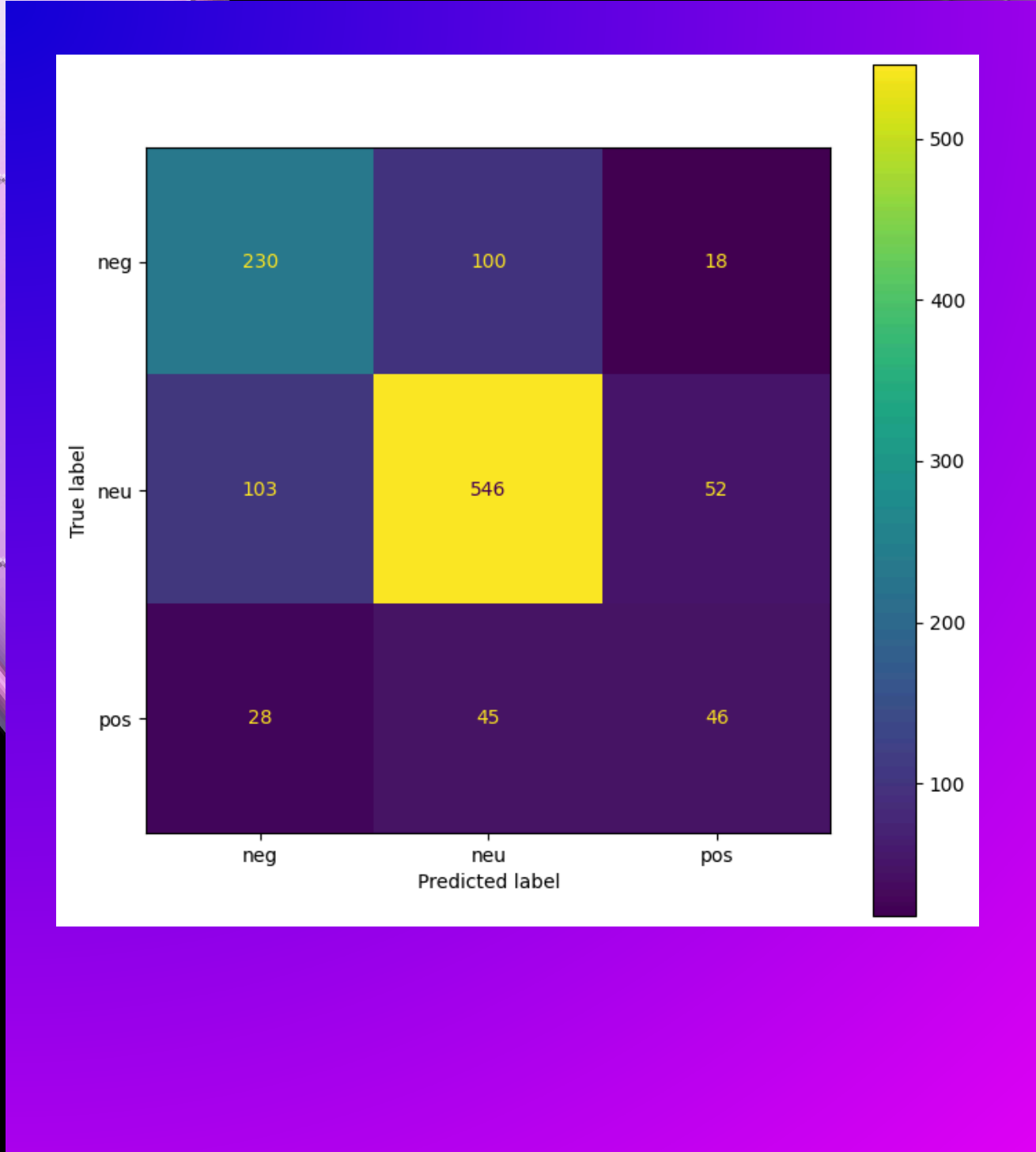
0.704

0.704

0.704

1168

=====



ไอเดียที่ 2

Page 09

Character n-gram

วัตถุประสงค์ของไอเดียนี้อคือ

เพื่อแก้ปัญหาคำสะกดผิด คำเพี้ยน และคำแสงที่พบบ่อยในข้อความแสดงความคิดเห็น
โดยใช้การแทนข้อความในระดับตัวอักษร เนื่องจากพีเจอร์ระดับคำอาจไม่สามารถจับรูปแบบ
เหล่านี้ได้อย่างมีประสิทธิภาพ

ตัดเป็น 3-gram
มากกก → 'มาก', 'ากก', 'กกก'

ตัดเป็น 4-gram/คำแสง
คำบบบบ → 'คำบ', 'บบบบ'

ตัดเป็น 5-gram
มากกก → 'มากกก'

สร้างฟีเจอร์ Character n-gram

Page 10

แนวคิด:

- ใช้ CountVectorizer แบบ character-level โดยกำหนดช่วง n-gram เป็น 3 ถึง 5 ตัวอักษร

```
# สร้าง CountVectorizer แบบ character-level
char_vectorizer = CountVectorizer(
    analyzer="char",    # วิเคราะห์ในระดับตัวอักษร ไม่ใช่ระดับคำ
    ngram_range=(3,5),  # ใช้ character n-gram ขนาด 3 ถึง 5 ตัวอักษร
    max_features=5000   # จำกัดจำนวนฟีเจอร์สูงสุด
)
```



ผลลัพธ์

=====

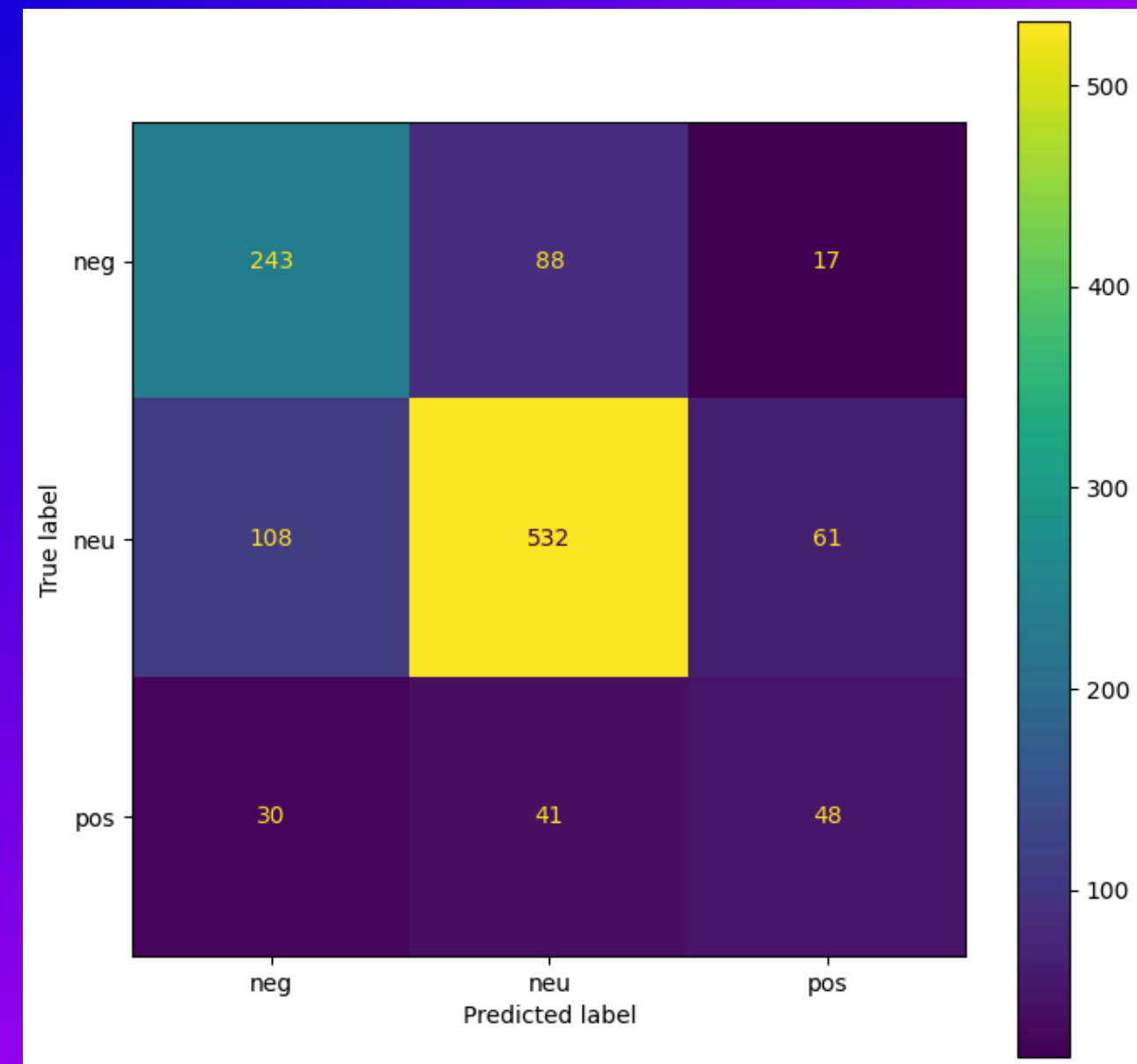
IDEA 2: Character n-gram (3-5)

Accuracy = 0.7046 | F1-score = 0.7074

Classification Report:

	precision	recall	f1-score	support
neg	0.638	0.698	0.667	348
neu	0.805	0.759	0.781	701
pos	0.381	0.403	0.392	119
accuracy		0.705		1168
macro avg	0.608	0.620	0.613	1168
weighted avg	0.712	0.705	0.707	1168

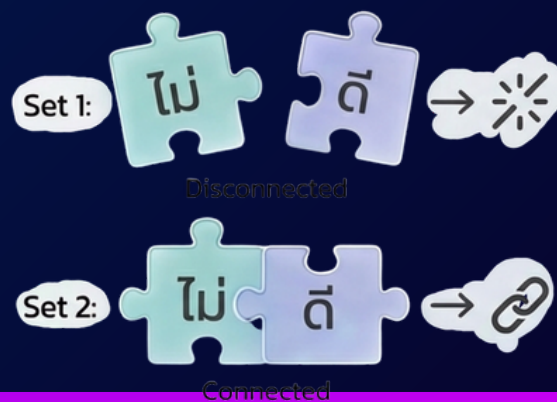
=====



Word n-gram (1,3)

วัตถุประสงค์ของไอเดียนี้คือ

เพื่อให้โมเดลสามารถเรียนรู้บริบทของข้อความในระดับวลี เนื่องจากอารมณ์ของข้อความมักเกิดจากการเรียงต่อกันของหลายคำ ไม่ใช่คำเดียว จึงขยายช่วง word n-gram จาก (1,2) เป็น (1,3) เพื่อจับความหมายเชิงอารมณ์ได้ดีขึ้น



การสร้างฟีเจอร์ word n-gram (1,3)

Page 13

แนวคิด:

- ขยายฟีเจอร์จากคำเดี่ยวเป็นวลี (unigram → trigram) เพื่อให้โมเดลเรียนรู้บริบทของข้อความ



```
# สร้าง CountVectorizer แบบ word-level
vectorizer_idea3 = CountVectorizer(
    tokenizer=lambda x: x.split("|"), # ใช้ tokenizer เดิมจาก preprocessing (แยกด้วย '|')
    ngram_range=(1,3),                # ใช้ unigram, bigram และ trigram
    max_features=5000,                # จำกัดจำนวนฟีเจอร์
    lowercase=False                    # ไม่แปลงเป็นตัวพิมพ์เล็ก เพราะ preprocess ไปแล้ว
)
```



ผลลัพธ์

=====

IDEA 3: ngram(1,3)

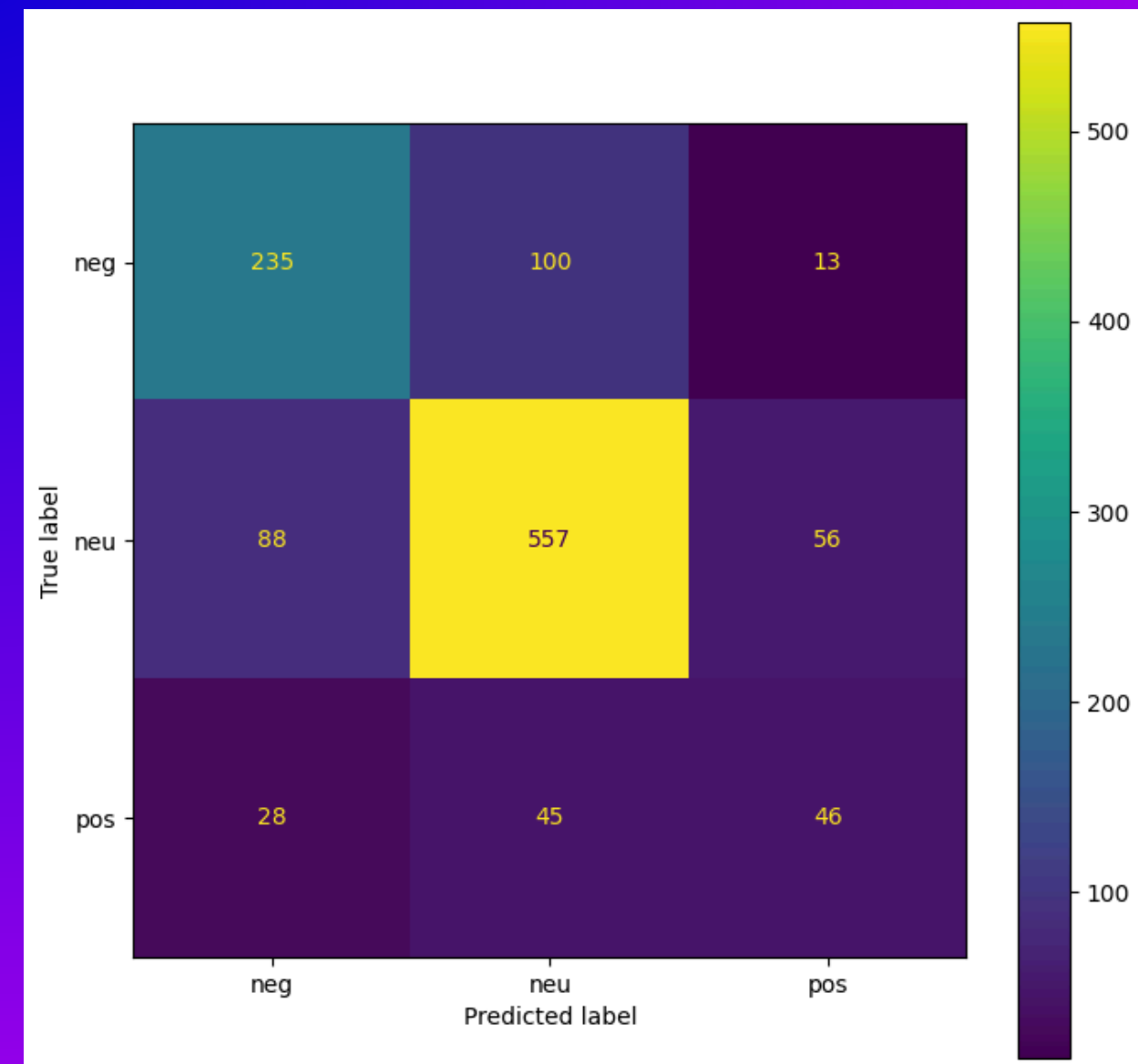
Accuracy = 0.7175 | F1-score = 0.7169

Classification Report:

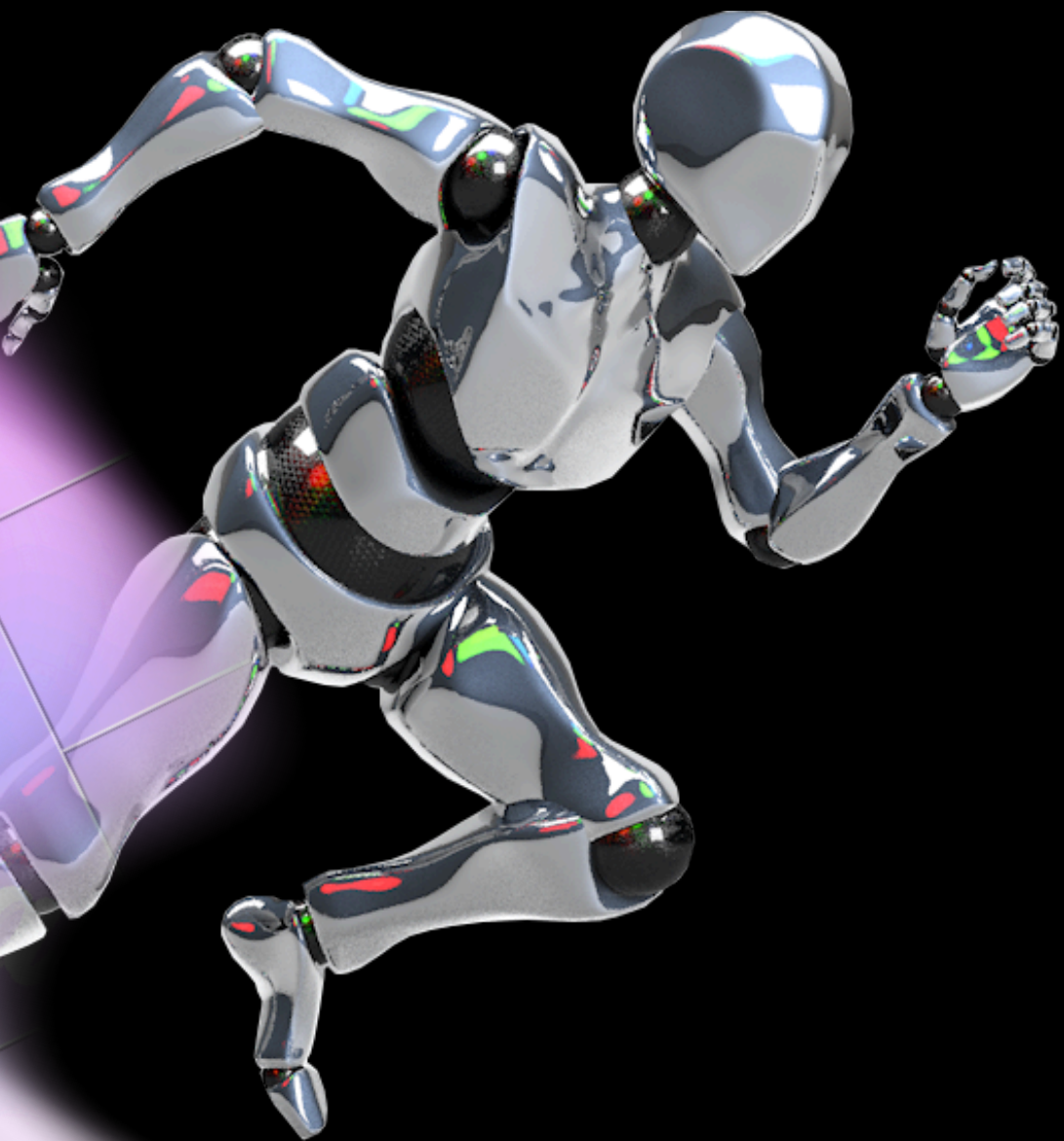
	precision	recall	f1-score	support
neg	0.670	0.675	0.672	348
neu	0.793	0.795	0.794	701
pos	0.400	0.387	0.393	119

accuracy			0.717	1168
macro avg	0.621	0.619	0.620	1168
weighted avg	0.716	0.717	0.717	1168

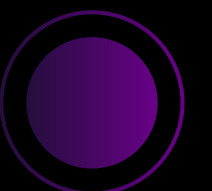
=====



ตารางเปรียบเทียบ



Experiment	Accuracy	F1-score
BASELINE	0.7038	0.7046
IDEA 1	0.7038	0.7040
IDEA 2	0.7046	0.7074
IDEA 3	0.7175	0.7169



THANK YOU

